# TS ACADEMY(Hajime Cohort) · CAPSTONE BRIEF Reputation & Feedback Intelligence Engine

## Submission Link:

https://docs.google.com/forms/d/e/1FAIpQLSfbWIX1B0MSjflltmWCBexfXQ9WfUDvhpCdOnblsZkuJKaKTw/viewform?usp=header

## The business

We run a multi-location car care and auto repair chain. Across our branches, we do good work, but our online reputation doesn't reflect it, because follow-up after a visit is inconsistent and depends on whoever's at the front desk that day. When something goes wrong, we usually find out the same way everyone else does: a bad review shows up online before anyone on our team has heard a word from that customer.

We also don't treat all feedback the same way right now, and we should. A customer who's mildly annoyed about a delay is not the same as a customer who's genuinely furious about a botched job, but our current (manual, inconsistent) process handles them identically: somebody reads it eventually and decides what to do.

We want a system that fixes both problems: it should treat every visit as an opportunity to collect feedback, read what customers actually say instead of just sorting them into two buckets, and put the right information in front of the right person at the right time. And because we operate multiple locations, our management team needs one place to see all of this happening, not five inboxes and a WhatsApp group.

## What we want built

### 1. Feedback collection

After a visit or job is marked complete, an automatic feedback request should go out to the customer. Keep this part simple. We just need a reliable trigger, based on the job being finished, that gets a request out to the customer asking how the visit went.

### 2. Sentiment scoring, not sorting

When the customer replies, don't just decide "happy" or "unhappy." Read what they actually wrote and score how positive or negative it is, and how severe. A customer who says "it was fine, a bit slow" and a customer who says "you people ruined my car and wasted my whole day" should not be treated the same, even though both are technically negative. We want the system to reflect that difference, not flatten it.

**3. Routing based on the score, not a keyword**

What happens next should depend on the score, not on whether the message contains a "bad" word:

- **Positive feedback** should be saved somewhere it can be turned into a public review later. We are not asking for it to be posted anywhere automatically. Every review a customer leaves goes through a human first. Store it as a "ready to post" item that someone on our team can open, read, and copy across to wherever we publish reviews, whenever they're ready to do that.
- **Negative feedback** should never be pointed toward a public review. It goes into a private queue instead, so we hear about problems before the internet does. ● The more severe the negative feedback, the faster we want to know. A mildly negative comment can sit in the queue for the team to handle in their own time. A severely negative one should alert a manager right away, not wait to be noticed.

**4. Repeat customers should be treated differently**

Before anything gets routed, check whether this customer has left negative feedback with us before. If this is their second time flagging a problem, don't wait for the sentiment score to decide how urgent it is. Escalate straight to a manager alert, regardless of how "mild" this particular message reads, because a pattern with one customer matters more than a single message.

**5. Don't leave the team facing a blank page**

For every piece of negative feedback, in addition to routing and alerting, generate a draft response addressed to that specific customer, referencing what they actually said. This is not a template with their name inserted. It should read like someone actually read their complaint. A human on our team reviews it, edits if needed, and sends it themselves. Nothing gets sent to a customer automatically.

**6. One dashboard for the whole picture**

Our management team should not have to piece this together from alerts and messages. We want a single dashboard where they can see:

- Every piece of feedback received, which location it came from, and its sentiment score ● What happened to it (routed to "ready to post," sitting in the private queue, or escalated to

a manager)
● The queue of positive reviews ready to be copied out and posted publicly ● Negative feedback alongside its AI-drafted response, and whether that response has been sent yet
● Which alerts have been acted on and which are still open
● A way to filter or search by location, so a manager at one branch isn't wading through every branch's feedback to find their own

**How the whole thing should flow, start to finish**

1. A visit gets marked complete.
2. The customer receives a feedback request.
3. The customer replies.
4. The reply gets read and scored for sentiment and severity, not just sorted into two buckets.
5. The system checks: has this customer left negative feedback with us before? 6. Based on the score and that history, the feedback is routed one of three ways: ○ Positive → saved to the "ready to post" queue for a human to publish later ○ Negative, first time or lower severity → private queue, draft response generated for the team to review and send
   ○ Negative, repeat customer or high severity → manager alerted immediately, draft response generated in parallel
7. Everything above, at every stage, is visible on the management dashboard, broken down by location.
8. If a reply comes in that the system genuinely can't make sense of, or can't confidently score, it should be flagged for a human to look at rather than guessed at and routed incorrectly.

## What "done" looks like

We're not expecting a connection to an actual public review platform for this. What we want to see is the full loop working end to end: a completed visit produces a feedback request, a reply comes back and gets scored (not just sorted), it's routed correctly based on both the score and the customer's history, a draft response exists for anything negative, a manager gets alerted when it's warranted, and all of it shows up correctly on the dashboard. Show us a positive case, a first-time negative case, and a repeat-negative case, so we can see all three paths actually behave differently, not just in theory.

## How to submit

The capstone is built as a group, but submission is done individually. Here's how that works:

1. One person from the group records the video, walking through what you built, explaining how it works, and running a live demo showing it actually working. It doesn't have to be

one person talking alone, the whole group can be on a call together while it's recorded, but only one recording is needed per group. Loom is preferred, but if you don't have access to Loom, any tool that gives you a shareable link works. If you can only do a normal screen recording, that's fine too, just download the file and upload it in step 3 so it has a link.

2. Take screenshots of the workflow and the dashboard.
3. Create one Google Drive folder for the group and upload into it:
    - The demo video (skip this if you already have a Loom link)
    - Screenshots of the workflow
    - Screenshots of the dashboard
4. Set the folder's sharing setting to "Anyone with the link," so it opens without anyone needing to request access.
5. Copy the folder link.
6. Whoever recorded the video (or the group lead) shares both links, the video link and the Google Drive folder link, with everyone else in the group.
7. Every single person in the group submits individually, using those same two links. A 10-person group means 10 separate submissions, all pointing to the same video and the same Drive folder.

**The video link is the most important part of this submission.** Anyone who doesn't submit one will lose their points for this capstone, no exceptions.

### Submission Link:

https://docs.google.com/forms/d/e/1FAIpQLSfbWIX1B0MSjflltmWCBexfXQ9WfUDvhpCdOnblsZkuJKaKTw/viewform?usp=header